# COGS 181 - Homework 3

due Tuesday, February 10, 2026 at 11:59PM

**Instructions:** Answer the questions below, attach your code, and insert figures to create a PDF file; submit your file via Gradescope. You may look up the information on the Internet, but you must write the final homework solutions by yourself.

**Late Policy:** 5% of the total points will be deducted on the first day past due. Every 10% of the total points will be deducted for every extra day past due.

**Problem 1. (14 points) Conceptual Questions**

**1.1** Is the following statement true or false?

Suppose the perceptron algorithm has computed a classifier parameterized by $\mathbf{w}$. Now, consider a new weight vector $\mathbf{w}' = c\mathbf{w}$ where $c$ is a nonzero scalar. Assume that the bias is zero. Using $\mathbf{w}'$ instead for classification might result in a different decision function.

**Solution:** True, if $c < 0$, the decision will be different since the sign of $\mathbf{w}' \cdot \mathbf{x}$ will be different.

**1.2 True or False.** For a two-layer neural network, if we increase the total number of neurons by adding one more hidden layer, then the test error of the new neural network can **always** be decreased.

**Solution:** False. There are a number of reasons why the test error may not decrease.

**1.3 True or False.** Suppose we design two alternative multi-layer neural network architectures to solve a classification problem. Let these two neural networks have the **same number of neurons**. If both are successfully trained on the same training set, then the deeper multi-layer neural network will always result in a smaller training error than the shallower one.

**Solution:** False. This is not always the case for a number of reasons.

**1.4 True or False.** The choice of the type of activation function in a multi-layer neural network affects how easily it can be trained **but not** the final training error of the classifier.

**Solution:** False. The choice of the type of activation function affects both the final training error and the difficulty in training.

**1.5 True or False.** The choice of the type of activation function in a multi-layer neural network decides the final training error of the classifier **but not** the difficulty in training.

**Solution:** False. The choice of the type of activation function affects both the final training error and the difficulty in training.

**1.6** Choose the **most** significant difference between the **backpropagation** process in multi-layer neural networks and the **training process** in the logistic regression algorithm:

A. deal with unsupervised learning vs. supervised learning

B. used with softmax function vs. sigmoid function

C. train networks with hidden layers vs. without hidden layers

D. used with gradient descent vs. closed-form solution

> **Solution:** C is the most significant difference. Logistic regression doesn't have hidden layers, while backpropagation is used to train networks with hidden layers.

**1.7** Kevin and John are having an argument about the backpropagation algorithm. Kevin claims that in backpropagation, first all the gradients with respect to weights in **all layers** are calculated and then all weights are updated simultaneously. John argues that the weights for **each layer** should be updated as soon as the gradients for the layer are computed, and then compute the gradients for the previous layer using updated weights. Which one of them is more favorable?

> **Solution:** Kevin is more favorable. You should compute the gradients for all layers at once, and then update the weights.

**Problem 2. (15 points) Back-Propagation in Neural Networks**

In this problem, we will derive necessary gradients for training the network of Figure 1 by backpropagation. **If you have already finished this question using Option 1 (the original form), you don't need extra work. If you have just started working on this problem, please take option 2, which is in a simpler form.**

**Option 1 (original form):** You can assume all variables are in matrix/vector form $\mathbf{x} \in \mathbb{R}^{m\times 1}$, $\mathbf{w}_1, \mathbf{w}_2 \in \mathbb{R}^{m\times h}$, $\mathbf{h}_1, \mathbf{h}_2 \in \mathbb{R}^{h\times 1}$, $\mathbf{h} \in \mathbb{R}^{2h\times 1}$, $\mathbf{w}_3, \mathbf{w}_4 \in \mathbb{R}^{2h\times n}$.

**Option 2 (simpler form [new]):** You can assume all variables are in vector/scalar form $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{w}_1 \in \mathbb{R}^m$, $\mathbf{w}_2 \in \mathbb{R}^m$, $\mathbf{h} \in \mathbb{R}^2$, $\mathbf{w}_3 \in \mathbb{R}^2$, $\mathbf{w}_4 \in \mathbb{R}^2$, $h_1$ and $h_2$ are scalars.

$$\sigma(\mathbf{w}, \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

**Hint:**

1. Use the chain rule to answer the questions in terms of the variables in the graph.
2. Use the gradient of the sigmoid function.

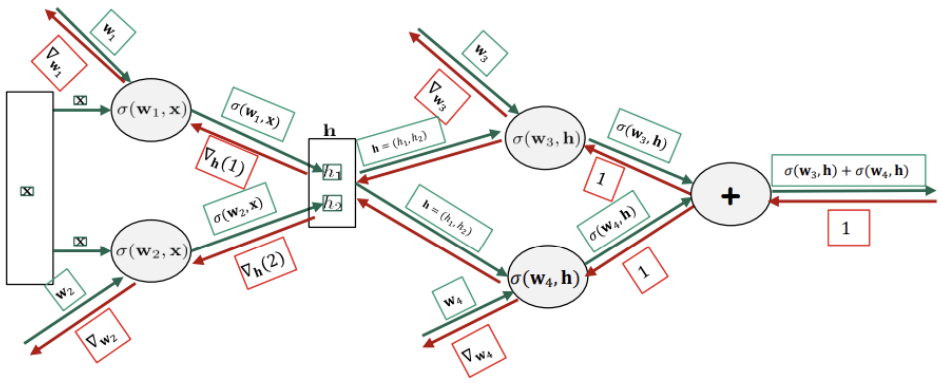



**Figure 1:** Question 4

**2.1 (3 points)** Compute $\nabla_{\mathbf{w}_4}$.

**Solution:** $\nabla_{\mathbf{w}_4} = \frac{\partial}{\partial \mathbf{w}_4}\sigma(\mathbf{w}_4, \mathbf{h}) = \sigma(\mathbf{w}_4, \mathbf{h})(1 - \sigma(\mathbf{w}_4, \mathbf{h}))\mathbf{h}$

**2.2 (3 points)** Compute $\nabla_{\mathbf{w}_3}$.

**Solution:** $\nabla_{\mathbf{w}_3} = \frac{\partial}{\partial \mathbf{w}_3}\sigma(\mathbf{w}_3, \mathbf{h}) = \sigma(\mathbf{w}_3, \mathbf{h})(1 - \sigma(\mathbf{w}_3, \mathbf{h}))\mathbf{h}$

**2.3 (3 points)** Compute $\nabla_{\mathbf{h}}$.

**Solution:** $\nabla_{\mathbf{h}} = \frac{\partial}{\partial \mathbf{h}}(\sigma(\mathbf{w}_4, \mathbf{h}) + \sigma(\mathbf{w}_3, \mathbf{h})) = \sigma(\mathbf{w}_4, \mathbf{h})(1 - \sigma(\mathbf{w}_4, \mathbf{h}))\mathbf{w}_4 + \sigma(\mathbf{w}_3, \mathbf{h})(1 - \sigma(\mathbf{w}_3, \mathbf{h}))\mathbf{w}_3$

**2.4 (3 points)** Compute $\nabla_{\mathbf{w}_2}$ in terms of $\nabla_{\mathbf{h}}(2)$

**Solution:** $\nabla_{\mathbf{w}_2} = \frac{\partial}{\partial \mathbf{w}_2}\sigma(\mathbf{w}_2, \mathbf{x}) \cdot \nabla_{\mathbf{h}}(2) = \sigma(\mathbf{w}_2, \mathbf{x})(1 - \sigma(\mathbf{w}_2, \mathbf{x}))\mathbf{x} \cdot \nabla_{\mathbf{h}}(2)$

**2.5 (3 points)** Compute $\nabla_{\mathbf{w}_1}$ in terms of $\nabla_{\mathbf{h}}(1)$

**Solution:** $\nabla_{\mathbf{w}_1} = \frac{\partial}{\partial \mathbf{w}_1}\sigma(\mathbf{w}_1, \mathbf{x}) \cdot \nabla_{\mathbf{h}}(1) = \sigma(\mathbf{w}_1, \mathbf{x})(1 - \sigma(\mathbf{w}_1, \mathbf{x}))\mathbf{x} \cdot \nabla_{\mathbf{h}}(1)$

**Problem 3. (7 points) Softmax Classification**

In this section, we consider the multi-class classification problem: We need to predict a K-way label $\hat{y} \in \{0, 1, ..., K-1\}$ for each input $\mathbf{x} = (x_0, ..., x_{D-1})^\top$. Note that we represent the $K$ classes by $\{0, 1, ..., K-1\}$ instead of $\{1, ..., K\}$ in lecture slides because it is easier to implement in Python. Then, we can express the conditional probability (or confidence) of class $j$ parameterized by $\mathtt{W} = (\mathbf{w}_0, ..., \mathbf{w}_{K-1})^\top \in \mathbb{R}^{K \times D}$ and $\mathbf{b} = (b_0, ..., b_{K-1})^\top \in \mathbb{R}^K$ as

$$p_j = p(y = j|\mathbf{x}) = \frac{e^{f_j}}{\sum_{k=0}^{K-1} e^{f_k}}$$

where $f_j = \mathbf{w}_j \mathbf{x} + b_j$.

We choose the class with the highest probability among all $K$ classes as the prediction $\hat{y}$:

$$\hat{y} = \text{argmax}_j p_j = \text{argmax}_j p(y = j|\mathbf{x})$$

In the training process, given a data point $\{(\mathbf{x}, y)\}$ in the training set, we firstly rewrite the label $y \in \{0, 1, ..., K-1\}$ in the one-hot encoding form $\mathbf{y} \in \mathbb{R}^K$:

$$\mathbf{y} = (y_0, ..., y_{K-1})^\top, \text{where } y_j = \begin{cases} 1 & \text{if } j = y \\ 0 & j \neq y \end{cases}$$

Then, we wish to optimize the following cross-entropy loss on the data point $\{(\mathbf{x}, y)\}$:

$$L(\mathbf{x}, y; \mathtt{W}, \mathbf{b}) = -\ln p_y = -\sum_{l=0}^{K-1} y_l \ln p_l$$

Given the training dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}$, we sum the cross-entropy losses over all data points in the dataset:

$$L(\mathtt{W}, \mathbf{b}) = \sum_{i=1}^{n} L(\mathbf{x}^{(i)}, y^{(i)}; \mathtt{W}, \mathbf{b})$$

In this problem, we will minimize $L(\mathtt{W}, \mathbf{b})$ using gradient descent and obtain the optimal $\mathtt{W}^*$ and $\mathbf{b}^*$.

Please derive the following:

**3.1 (5 points)** For one data point $\{(\mathbf{x}, y)\}$, show that the gradient of $L(\mathbf{x}, y; \mathtt{W}, \mathbf{b})$ with respect to the $f_j$ is given by:

$$\frac{\partial L(\mathbf{x}, y; \mathtt{W}, \mathbf{b})}{\partial f_j} = p_j - y_j$$

**Solution:** Let $L = -\sum_{l=0}^{K-1} y_l \ln p_l$, $Z = \sum_{k=0}^{K-1} e^{f_k}$, and $\delta_{lj} = \mathbf{1}(l = j)$. Then,

$$\begin{aligned}\frac{\partial L}{\partial f_j} &= -\sum_{l=0}^{K-1} y_l \frac{\partial \ln p_l}{\partial f_j} = -\sum_{l=0}^{K-1} y_l \left(\delta_{lj} - \frac{e^{f_j}}{Z}\right) = -\sum_{l=0}^{K-1} y_l(\delta_{lj} - p_j) \\ &= -y_j + p_j \sum_{l=0}^{K-1} y_l = -y_j + p_j = p_j - y_j\end{aligned}$$

**3.2 (2 points)** Assume $\mathbf{w}_j = (w_{j0}, ..., w_{j(D-1)})^\top$, show that the gradient of $L(\mathbf{x}, y; \mathtt{W}, \mathbf{b})$ with respect to the $w_{jd}$ and $b_j$ is given by:

$$\frac{\partial L(\mathbf{x}, y; \mathtt{W}, \mathbf{b})}{\partial w_{jd}} = (p_j - y_j)x_d \qquad \frac{\partial L(\mathbf{x}, y; \mathtt{W}, \mathbf{b})}{\partial b_j} = p_j - y_j$$

**Solution:**

$$\frac{\partial L}{\partial w_{jd}} = \frac{\partial L}{\partial f_j} \cdot \frac{\partial f_j}{\partial w_{jd}} = (p_j - y_j) \cdot x_d$$

$$\frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial f_j} \cdot \frac{\partial f_j}{\partial b_j} = (p_j - y_j) \cdot 1 = p_j - y_j$$

**Problem 4. (12 points) Feed-Forward Neural Network**

In this question, on top of the single-layer network, we add another layer between the input and output with the sigmoid activation function, and this hidden layer contains 3 **hidden units**. Assume an input feature $\mathbf{x} \in \mathbb{R}^{2\times 1}$. Thus, the probability of output $y$ is represented as:

$$p(y|\mathbf{x}) = \sigma\left(y \times (\mathbf{w}_2^\top \sigma(\mathtt{W}_1^\top \mathbf{x} + \mathbf{b}_1) + b_2)\right)$$

where $\mathtt{W}_1 \in \mathbb{R}^{2\times 3}$ and $\mathbf{w}_2 \in \mathbb{R}^{3\times 1}$ are the weights, $\mathbf{b}_1 \in \mathbb{R}^{3\times 1}$ and $b_2 \in \mathbb{R}^{1\times 1}$ are the biases. The activation function is the logistic sigmoid function, which is defined as follows:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

The logistic sigmoid function is applied on each entry in the vector or matrix. For example, we define $\mathbf{h} = \sigma(\mathtt{W}_1^\top \mathbf{x} + \mathbf{b}_1)$ as the output of the first layer. The output of $\mathtt{W}_1^\top \mathbf{x} + \mathbf{b}_1$ is a 3-dimension vector, and the sigmoid function $\sigma(\cdot)$ is applied at each entry of the 3-dimension vector, so $\mathbf{h}$ is still a 3-dimension vector (i.e. $\mathbf{h} \in \mathbb{R}^{3\times 1}$).

In order to train a binary classifier with this network, we define a cross-entropy loss function to measure the distance between the target and the prediction from this network, and the loss function $L$ for one point $(\mathbf{x}, y)$ is defined as:

$$L(\mathtt{W}_1, \mathbf{w}_2, \mathbf{b}_1, b_2) = -\ln p(y|\mathbf{x})$$

In the training process, we aim to minimize the loss function $L_{\text{train}}$ over the training set $S_{\text{train}} = \{(\mathbf{x}^{(0)}, y^{(0)}), ..., (\mathbf{x}^{(n-1)}, y^{(n-1)})\}$:

$$L_{\text{train}}(\mathtt{W}_1, \mathbf{w}_2, \mathbf{b}_1, \mathbf{b}_2) = -\sum_{i=0}^{n-1} \ln p(y^{(i)}|\mathbf{x}^{(i)})$$

Since there is no closed-form solution for the weights and the bias in the network, we consider a gradient descent algorithm to train the network.

Assume $p = p(y|\mathbf{x})$, please prove the following:

**4.1** $\dfrac{\partial L}{\partial \mathbf{w}_2} = (p-1)y \times \mathbf{h}$

**Solution:** Let $z = y \times (\mathbf{w}_2^\top \mathbf{h} + b_2)$ so $p = \sigma(z)$. Then $\frac{\partial z}{\partial \mathbf{w}_2} = y\mathbf{h}$ and

$$\frac{\partial L}{\partial z} = -\frac{\partial}{\partial z} \ln p = -\frac{1}{p} \cdot p(1-p) = p - 1$$

$$\frac{\partial L}{\partial \mathbf{w}_2} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mathbf{w}_2} = (p-1) \cdot y \cdot \mathbf{h}$$

**4.2** $\dfrac{\partial L}{\partial b_2} = (p-1)y$

**Solution:** Using the same $z = y \times (\mathbf{w}_2^\top \mathbf{h} + b_2)$, we have $\frac{\partial z}{\partial b_2} = y$. Hence,

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b_2} = (p-1) \cdot y$$

**4.3** $\dfrac{\partial L}{\partial \mathbf{h}} = (p-1)y \times \mathbf{w}_2$

**Solution:** Using the same $z = y \times (\mathbf{w}_2^\top \mathbf{h} + b_2)$, we have $\frac{\partial z}{\partial \mathbf{h}} = y\mathbf{w}_2$. Hence,

$$\frac{\partial L}{\partial \mathbf{h}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \mathbf{h}} = (p-1) \cdot y \cdot \mathbf{w}_2$$

**Problem 5. (12 points) 2-D Convolution**

Suppose we have a small network for image recognition. The input image $\mathbf{x} \in \mathbb{R}^{4\times4}$. The network has three layers named Conv, MaxPool, and ReLU:

$$\text{Image}\mathbf{x} \to \text{Conv} \xrightarrow{\mathbf{z}_1} \text{MaxPool} \xrightarrow{\mathbf{z}_2} \text{ReLU} \to \mathbf{z}_3$$

where $\mathbf{z}_1, \mathbf{z}_2$ are intermediate output and $\mathbf{z}_3$ is the final output. The details of the layers are given below:

**Conv:** A single-channel convolution layer with 3×3 kernel, stride size 1 and no padding.

**MaxPool:** A max pooling layer with 2×2 kernel and stride size 2.

**ReLU:** A ReLU activation layer which has $\max(0, z)$ as the output of input $z$.

**5.1 (6 points) Output Shape Computation**

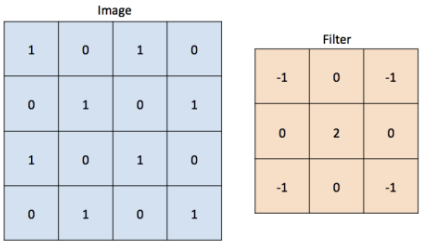
Please calculate the shape of the outputs and fill in the blanks below:

1. Shape of input $\mathbf{x}$: $4 \times 4$.
2. Shape of intermediate output $\mathbf{z}_1$: _____ × _____ .
3. Shape of intermediate output $\mathbf{z}_2$: _____ × _____ .
4. Shape of final output $\mathbf{z}_3$: _____ × _____ .

**Solution:**

1. Shape of input $\mathbf{x}$: $4 \times 4$.
2. Shape of intermediate output $\mathbf{z}_1$: $2 \times 2$.
3. Shape of intermediate output $\mathbf{z}_2$: $1 \times 1$.
4. Shape of final output $\mathbf{z}_3$: $1 \times 1$.

**5.2 (6 points) Output Value Computation**




Assume the values of input image $\mathbf{x}$ and the filter of the Conv layer are given in the figure above. Please compute the values of outputs $\mathbf{z}_1$, $\mathbf{z}_2$ and $\mathbf{z}_3$ by hand.

**Hint:** The values of $\mathbf{z}_1$, $\mathbf{z}_2$ and $\mathbf{z}_3$ should match the shapes in the previous sub-problem.

**Solution:**

$$\mathbf{z}_1 = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix}$$
$$\mathbf{z}_2 = \max(z_1) = \max(0, -2) = 0$$
$$\mathbf{z}_3 = \max(0, 0) = 0$$

**Programming Part**

Question 6 and 7 are coding questions. Please refer to the Jupyter notebook `hw3.ipynb`.

**Problem 6. (20 points) Softmax Classification**

**Problem 7. (20 points) Feed-Forward Neural Network**